

Exam Medicine  By CU Updates

Database Management Systems

Course Code: 25CSH-245

UNIT 1

MST-1 SMART STUDY GUIDE

Chapter 1: Introduction to Databases & DBMS

Chapter 2: Data Models (including ER Modeling)

Chapter 3: Relational Algebra & Relational Calculus

Premium Exam-Focused Study Notes for Computer Science Students

Designed for Maximum Marks, Deep Conceptual Understanding, and Efficient Study Time

Compiled by Ankul kumar & Team CU Updates
Date: August 2026

Contents

1	Introduction to Databases and DBMS	4
1.1	Basic Database Concepts	4
1.1.1	Data vs. Information	4
1.1.2	What is a Database?	4
1.1.3	Characteristics of a Database	4
1.2	Database Management System (DBMS)	5
1.2.1	Functions of a DBMS	5
1.2.2	Traditional File Systems vs. DBMS	6
1.3	Database System Architecture Tiers	6
1.3.1	1-Tier Architecture	6
1.3.2	2-Tier Architecture	7
1.3.3	3-Tier Architecture	7
1.4	Three-Level ANSI-SPARC Architecture	7
1.4.1	Database Mappings	8
1.5	Data Independence	8
1.6	Database Administrator (DBA)	9
1.6.1	Roles and Responsibilities of a DBA	9
1.7	Relational Data Structure	10
1.7.1	Concepts of Relations, Attributes, and Domains	10
1.7.2	Formal Properties of a Relation	10
1.8	Database Schema vs. Database Instance	11
1.9	Keys in Relational Databases	11
1.9.1	Definitions and Structural Rules	11
1.10	Integrity Constraints	12
1.10.1	Entity Integrity Constraint	12
1.10.2	Referential Integrity Constraint	12
1.10.3	Referential Actions on Delete and Update	12
2	Data Models	13
2.1	Overview and Comparison of Data Models	13
2.2	The Entity-Relationship (ER) Model	13
2.2.1	Entity and Entity Sets	13
2.2.2	Attributes and Classifications	13
2.2.3	Relationships and Relationship Sets	14
2.3	Mapping Constraints	14
2.3.1	Cardinality Ratios	14
2.3.2	Participation Constraints	14
2.4	Weak Entity Sets	14
2.5	ER Diagram Basic Symbols Reference	15
2.6	ER Model Design Issues	15

3 Relational Algebra and Relational Calculus 16

3.1 Relational Algebra: Syntax, Semantics, and Basic Operators	16
3.1.1 Unary Operators	16
3.1.2 Binary Set Operators	17
3.2 Additional Relational Algebra Operators	17
3.2.1 Join Operations	17
3.2.2 Division Operator ($\div$)	18
3.2.3 Grouping and Aggregation Operator (γ)	18
3.3 Introduction to Relational Calculus	18
3.3.1 Logic Operators and Quantifiers Key	18
3.3.2 Tuple Relational Calculus (TRC)	18
3.3.3 Domain Relational Calculus (DRC)	19
3.3.4 Safety of Expressions	19
3.4 Computational Capabilities and Codd's Theorem	19

4 Master Formula & Rule Sheet 20

4.1 Core Structural Properties	20
4.2 Relational Algebra Operators Cheat Sheet	20
4.3 Database Integrity Constraint Rules	20

5 Solved Problem Manual 21

5.1 Problem Category A: Key Identification & Cardinality Analysis	21
5.2 Problem Category B: Relational Algebra Query Construction	21
5.2.1 Query 1: Retrieve names and departments of all students older than 20. .	22
5.2.2 Query 2 (Nested Queries/Aggregation): Find departments where average student age is greater than 21.	22
5.2.3 Query 3 (Division Operator): Find the RollNo of students who have enrolled in EVERY course offered.	22
5.3 Problem Category C: Relational Calculus Query Construction	23
5.3.1 Tuple Relational Calculus (TRC) Formulation:	23
5.3.2 Domain Relational Calculus (DRC) Formulation:	23
5.4 Problem Category D: Referential Integrity Cascades	23

6 Expected Exam Question Bank 25

6.1 Section A: High-Yield 2-Mark Questions	25
6.1.1 Q1: Define a Database Management System (DBMS). [59]	25
6.1.2 Q2: Differentiate between a Database Schema and a Database Instance. [59]	25
6.1.3 Q3: State the minimality property of a Candidate Key. [27]	25
6.1.4 Q4: What is the purpose of the Entity Integrity Constraint? [30]	25
6.1.5 Q5: Explain why Physical Data Independence is easier to achieve than Logical Data Independence. [11]	25
6.1.6 Q6: What does the Division operator ($\div$) in Relational Algebra represent? [48]	26
6.1.7 Q7: Define a Weak Entity Set. Give one real-world example. [39]	26
6.1.8 Q8: Under what condition are two relations Union-Compatible? [46] ...	26
6.2 Section B: Synthesis & Design 5-Mark Questions	26
6.2.1 Q1: Explain the ANSI-SPARC Three-Level Architecture. Discuss how it supports Data Independence. [62]	26
6.2.2 Q2: Describe the Entity-Relationship (ER) Model attribute classifications. Give clear examples of each. [36]	26
6.2.3 Q3: Compare the Relational, Hierarchical, and Network Data Models. [63]	27
6.3 Section C: Long-Answer & Problem-Solving Questions	27

6.3.1 Q1: Consider the relational schema:	27
7 1-Hour MST Revision Sheet	28
7.1 TOP 10 MUST-PREPARE QUESTIONS	28
7.2 Final Core Concept Checklist	28

CU UPDATES

Chapter 1

Introduction to Databases and DBMS

1.1 Basic Database Concepts

1.1.1 Data vs. Information

A fundamental prerequisite to database management is distinguishing between raw values and their contextual interpretations.

- ◁ **Data** refers to raw, unorganized, and uninterpreted facts, figures, or observations that do not possess intrinsic meaning when viewed in isolation [3].
- ◁ **Information** is processed, structured, and organized data presented in a context that makes it meaningful, logical, and directly useful to decision-makers [3].

Table 1.1: Technical Comparison of Data vs. Information [3]

Feature	Data	Information
Definition	Raw facts and figures without context [3].	Processed, structured, and contextual data
Meaning	Lacks intrinsic meaning in isolation [3].	Meaningful, conveying clear understanding
Organization	Unstructured and random [4].	Highly structured and ordered.
Utility	Cannot be directly used for decision-making.	Serves as a primary aid for decision-making
Example	101, Rahul, CSE, 8.5 [3]	Rahul (Roll No: 101) belongs to CSE; CGP

1.1.2 What is a Database?

A **Database** is an organized, logical, and structured collection of physically or logically related data, stored electronically in a digital medium and designed to be accessed, managed, queried, and updated efficiently [4]. In modern relational architectures, this data is logically structured into two-dimensional tables consisting of rows and columns [4].

1.1.3 Characteristics of a Database

According to the foundational properties of database systems, a database exhibits several core characteristics:

- ◁ **Shared Access:** Data is simultaneously accessible to multiple users and applications across a network [5, 13].

- ◁ **Data Persistence:** Data remains stored permanently even after system shut-downs, program terminations, or physical power failures [5].
- ◁ **Controlled Redundancy:** Duplicate storage of identical data items is minimized via normalization [4, 12]. 5
- ◁ **Data Consistency:** Ensures identical data values are uniform across all system access points [5, 13].
- ◁ **Enforced Integrity:** Data values must comply with defined real-world constraints (such as data types or keys) [5, 14].
- ◁ **Controlled Security:** Restricts unauthorized view or modification of sensitive data tables [5, 13].

REMEMBER

Data Redundancy vs. Data Inconsistency: Data Redundancy is the physical storage of redundant copies of the same data across multiple files [6, 12]. Data Inconsistency is the direct consequence of uncontrolled redundancy: when a change is made to one copy of the data but not to others, the system contains conflicting facts [6, 13]. Centralized database storage mitigates this by keeping a single, canonical source of truth [7, 12].

1.2 Database Management System (DBMS)

A **Database Management System (DBMS)** is a specialized, comprehensive software package that serves as a programmatic interface between application programs/end-users and the underlying physical database files [5]. It provides systemic tools to define, create, query, maintain, and control access to the database [5].

1.2.1 Functions of a DBMS

A modern DBMS performs several critical functions to manage database systems:

1. **Data Dictionary Management:** Stores definitions of the data elements and their relationships (metadata).
 2. **Data Storage and Retrieval Management:** Creates and manages complex physical structures required for data storage [6].
 3. **Security Management:** Administers user authentication, roles, and fine-grained data access permissions [6, 13].
 4. **Multi-user Concurrency Control:** Coordinates simultaneous data updates by multiple transactions to ensure data consistency and prevent conflicts [6, 13].
 5. **Backup and Recovery Management:** Automates data duplication schedules and recovers data from physical crashes or failures [6, 14].
 6. **Data Integrity Management:** Automatically validates data inputs against predefined integrity constraints [6, 14].
-

1.2.2 Traditional File Systems vs. DBMS

To appreciate why database management systems are used, one must study the challenges of traditional, flat file systems where data is managed in separate, isolated files by different department applications [6].

Table 1.2: Detailed Contrast of Traditional File Systems vs. DBMS [6, 7]

Metric	Traditional File System	Database Management System
Data Redundancy	High. Redundant copies are stored by separate departments [6, 12].	Minimized. Centralized logical schema ensures single storage [7, 12].
Data Consistency	Poor. Inconsistent updates to isolated files cause data mismatch [6].	High. Global updates immediately propagate to all logical views [13].
Concurrency Control	Lacking. Simultaneous access causes file lock conflicts or data corruption [6].	Robust. Employs advanced transactional protocols to allow parallel access [6, 13].
Security Enforcement	Simple. Rely on operating system file permissions; lack granular row/column access controls [6].	Advanced. Granular access rights, logical views, and database schemas [13].
Data Dependence	High. Application program logic is heavily coupled with physical file structure [15, 16].	Low. Data abstraction guarantees independence at logical and physical layers [9, 15].
Query Processing	Difficult. Custom application programs must be written to search data [6].	Efficient. Uses standard, declarative query languages (like SQL) [20].

COMMON TRAP

Avoid Absolute Statements in Exams!

Do not write that traditional file systems have “*zero security*” or “*cannot support any concurrent access*.” Instead, write that file systems typically rely on simple, coarse-grained operating system file permissions which lack granular, database-level security controls (e.g., protecting specific columns or rows) and lack automated transaction mechanisms for concurrent access [6, 13].

1.3 Database System Architecture Tiers

Database system architectures dictate how client devices communicate physically and logically with the database server. They are categorized based on the logical separation of presentation, application, and database functions.

1.3.1 1-Tier Architecture

In a 1-Tier architecture, the database files, the DBMS software, the business application, and the user interface reside entirely on a single physical machine [8]. This is common for local standalone applications and single-user systems (e.g., MS Access used on a single PC) [8].

1.3.2 2-Tier Architecture

A 2-Tier architecture is a classic client-server model [8]. The client device hosts the user interface and application programs (presentation and logic), while the database server hosts the DBMS and the physical database files [8]. Communication occurs directly between client and server via direct database connection drivers (such as ODBC or JDBC) [8].

1.3.3 3-Tier Architecture

A 3-Tier architecture introduces an intermediate layer between the client and the database server [9].

- ◁ **Presentation Tier (Client):** Handles the user interface and visual display (e.g., web browsers, thin clients) [9].
- ◁ **Application Tier (Business Logic):** Implements business rules and processes logic [9]. It acts as a middleman, translating client requests into database queries and processing the returned results [9].
- ◁ **Database Tier (DBMS):** Manages the database storage, queries, and transactions [9].

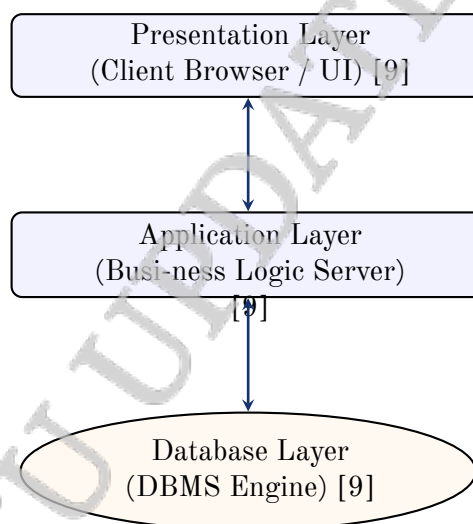


Figure 1.1: 3-Tier Database System Architecture [9]

EXAM FOCUS

Why is 3-Tier standard for web applications?

In web database designs, a 3-Tier architecture is highly preferred [9]. It prevents the client from directly connecting to the database server [9]. By routing all queries through the business logic server, it improves database security, isolates database structures, and makes the system highly scalable [9].

1.4 Three-Level ANSI-SPARC Architecture

Proposed by the Standards Planning and Requirements Committee (SPARC) of the American National Standards Institute (ANSI), this architecture defines a systematic framework to manage database abstraction and isolate user applications from physical database structures [9].

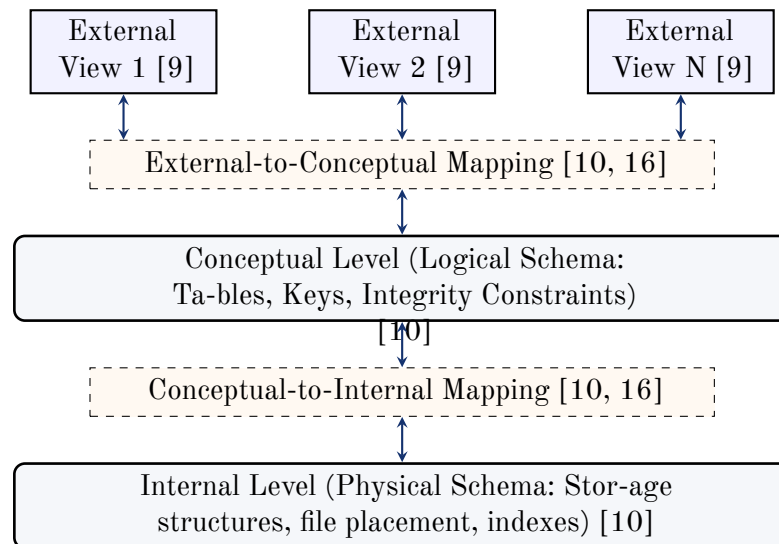


Figure 1.2: ANSI-SPARC Three-Level Database Architecture [9]

The architecture is divided into three distinct levels of database abstraction:

1. **External Level (View Level):** The highest level of abstraction [9]. It describes how individual users or client applications view specific subsets of data [9]. Multiple external views can exist simultaneously [9]. A student, for instance, only accesses their personal transcript, while a registrar views the entire department enrollment history [10].
2. **Conceptual Level (Logical Level):** The middle level of abstraction [10]. It describes the logical structure of the entire database [10]. It defines all entities, their attributes, relationships, data types, and logical integrity constraints [10]. It is completely independent of both how the data is physically stored and how individual users view it [10].
3. **Internal Level (Physical Level):** The lowest level of database abstraction [10]. It describes how the database is physically stored on disk [10]. It specifies file layouts, block storage sizes, page allocations, record placement strategies, and performance indexes [10].

1.4.1 Database Mappings

The separation of levels is maintained dynamically via runtime mappings:

- ◁ **External-to-Conceptual Mapping:** Maps a specific external view to the central conceptual schema [10, 16]. It translates external queries (expressed over specific view attributes) into queries over logical tables [10, 16].
- ◁ **Conceptual-to-Internal Mapping:** Maps logical conceptual tables to the underlying physical storage structures [10, 16]. It translates logical schema updates into physical page write commands [10, 16].

1.5 Data Independence

Data Independence is the ability to modify the database schema at one level of the ANSISPARC architecture without forcing a rewrite of the schema at the next higher level or disrupting application programs [15].

Table 1.3: Technical Comparison of Logical vs. Physical Data Independence [10, 11, 16]

Dimension	Logical Data Independence	Physical Data Independence
Definition	Ability to modify the conceptual schema without altering external views or application code [10, 16].	Ability to modify the internal schema without altering the logical conceptual schema [11, 16].
Mapping Layer	Supported by the External-to-Conceptual Mapping layer [10, 16].	Supported by the Conceptual-to-Internal Mapping layer [10, 16].
Difficulty	Harder to achieve because logical structure changes directly impact user query definitions [17].	Easier to achieve because logical schemas are completely isolated from physical storage [11].
Examples	Adding a new logical attribute to a table; splitting a relation into two normalized schemas [17].	Creating a B+ tree index; migrating storage files; changing physical block sizes [11, 16].

EXAM FOCUS

Why is physical data independence easier to achieve than logical data independence?

Physical data independence is easier because internal changes (such as creating an index or re-organizing file clustering) only affect query execution performance, leaving the logical column and table names completely unchanged [11, 16]. Logical data independence is much harder to achieve because changing logical tables (e.g., splitting a table into two during normalization) alters the exact names and logical structures that application queries reference, requiring complex mapping adjustments to keep application logic functional [17].

1.6 Database Administrator (DBA)

The **Database Administrator (DBA)** is the central technical authority or administrative role responsible for managing, maintaining, tuning, and securing the database system [17]. Think of the DBA as the technical custodian and guardian of the entire database infrastructure [18].

1.6.1 Roles and Responsibilities of a DBA

- ◁ **Schema Definition:** Translates physical requirements into conceptual tables, schemas, and relational structures [18].
- ◁ **Access and Security Management:** Controls database access permissions, creates user accounts, assigns roles, and prevents unauthorized data breaches [18].
- ◁ **Data Integrity Control:** Establishes and monitors relational constraints to prevent invalid data entry [18].
- ◁ **Performance Monitoring and Tuning:** Monitors query speeds, optimizes indexing structures, and tunes physical allocations [18, 19].
- ◁ **Backup and Recovery Management:** Designs robust backup policies and restores databases in the event of hardware crashes [19].
- ◁ **Concurrency Control Coordination:** Resolves database deadlocks and monitors transactional lock queues [19].

1.7 Relational Data Structure

Formulated by IBM researcher **E. F. Codd**, the Relational Data Model represents all database data logically as independent, two-dimensional tables called **Relations** [20].

1.7.1 Concepts of Relations, Attributes, and Domains

To master relational database theory, students must learn the formal mathematical terminology:

- ◁ **Relation (R):** A table structure consisting of a header (attributes) and a body (tuples) [21]. Mathematically, a relation R is a subset of the Cartesian product of one or more attribute domains [21]: $R \subseteq D_1 \times D_2 \times \dots \times D_n$
- ◁ **Tuple (t):** A single row within a relation representing a specific, unique record instance [21].
- ◁ **Attribute (A):** A named column in a relation representing a specific property or characteristic of an entity [22, 23].
- ◁ **Domain (D):** A set of permitted, atomic (indivisible) values for a given attribute [24]. For example, the domain of a CGPA attribute is restricted to real numbers between 0.0 and 10.0 [24].
- ◁ **Degree:** The total number of columns (attributes) in a relation [22].
- ◁ **Cardinality:** The total number of rows (tuples) currently stored in a relation [22].

COMMON TRAP

Degree vs. Cardinality:

Never mix these two properties in exams! Degree counts Columns (attributes), which are structurally fixed and change very rarely [22, 26]. Cardinality counts Rows (tuples), which change dynamically with every record insertion or deletion [22, 26].

1.7.2 Formal Properties of a Relation

A true relation must satisfy the following fundamental properties:

1. **No Duplicate Tuples Allowed:** In set theory, a relation is a set of tuples; therefore, every tuple in a relation must be unique [23].
2. **Order of Tuples is Irrelevant:** Changing the physical order of rows does not alter the logical meaning of the relation [23].
3. **Order of Attributes is Irrelevant:** Columns are identified by name, so their visual placement order is logically meaningless [23].
4. **Attribute Values are Strictly Atomic:** Every value in a row-column intersection must be a single, indivisible value [23]. Multi-valued attributes or nested structures are prohibited in a relation [23, 24].

1.8 Database Schema vs. Database Instance

A database undergoes a clear separation between its structural definition and its physical state:

- ◁ **Database Schema:** The overall logical layout, design blueprint, and integrity constraints of the database [25]. It is defined structurally once during database creation and changes very rarely [25, 26]. 11
- ◁ **Database Instance:** The actual data stored in the database at a specific snapshot in time [25, 26]. It changes frequently through daily transactions [26]. Table 1.4: Technical Contrast of Database Schema vs. Instance [26]

Metric	Database Schema	Database Instance
Concept	Structural logical blueprint [25].	Actual data content snapshot [25, 26].
Stability	Highly stable; changes very rarely [26].	Highly dynamic; changes with every insert/delete [26].
Design Phase	Defined during logical planning phase [25].	Populated during runtime system operations [26].
Analogy	Building blueprint [25].	The occupants living inside the building [26].

1.9 Keys in Relational Databases

Keys are attributes used to uniquely identify tuples in a relation, establish relationships between tables, and enforce data integrity constraints [27].

1.9.1 Definitions and Structural Rules

- ◁ **Super Key:** A set of one or more attributes that, taken together, uniquely identify tuples in a relation.
- ◁ **Candidate Key:** A minimal Super Key [27]. It is an attribute or a set of attributes that uniquely identifies a tuple in a relation [27], satisfying:
 1. **Uniqueness:** No two tuples can share identical values for the candidate key [27].
 2. **Minimality:** No proper subset of the candidate key attributes can uniquely identify the tuples [27].
- ◁ **Primary Key:** The specific candidate key chosen by the database designer to uniquely identify tuples in a relation [28]. It cannot contain **NULL** values and must remain stable [28].
- ◁ **Foreign Key:** An attribute (or set of attributes) in one relation (referencing relation) that references a candidate key (typically the primary key) of another relation (referenced relation) [29].

MUST KNOW**NULL Rules for Relational Keys:**

- ◁ **Primary Keys:** Strictly prohibited from containing NULL values under any circumstances (enforced by Entity Integrity) [28, 30].
- ◁ **Candidate Keys:** By default, candidate keys that are not selected as the primary key can technically contain NULL values (if the schema lacks a NOT NULL constraint), provided the unique constraint is maintained.
- ◁ **Foreign Keys:** Fully permitted to contain duplicate values and NULL values (signifying an optional relationship where the referencing tuple has no associated referenced tuple) [29].

1.10 Integrity Constraints

Integrity constraints are structural rules enforced by the DBMS to prevent unauthorized modifications, keep data valid, and ensure data consistency [30].

1.10.1 Entity Integrity Constraint

Enforces that no primary key column can contain a NULL value and every primary key value must be unique [30]. This guarantees that every individual entity remains uniquely identifiable across the database system [30].

1.10.2 Referential Integrity Constraint

Enforces that a foreign key value in a referencing relation must either:

1. Be completely NULL [31].
2. Or must match an existing primary key value in the referenced relation [31].

This constraint connects two relations and prevents the creation of "orphan records" [31, 32].

1.10.3 Referential Actions on Delete and Update

When a user attempts to delete or update a primary key value in a referenced relation that is referenced by rows in another relation, the DBMS must handle the foreign key columns according to defined actions:

- ◁ **CASCADE:** Automatically deletes or updates the corresponding referencing rows [32, 33].
- ◁ **SET NULL:** Sets the referencing foreign key columns to NULL [32].
- ◁ **RESTRICT / NO ACTION:** Blocks the deletion or update of the referenced primary key row [32].

Chapter 2

Data Models

2.1 Overview and Comparison of Data Models

A Data Model is a structural conceptual framework used to describe data representation, relationships, and integrity rules [33, 34].

Table 2.1: Rigorous Technical Comparison of Traditional Data Models [43]

Metric	Hierarchical Model [43]	Network Model [43]	Relational Model [43]
Logical Design	Tree-structured parent-child hierarchy	Graph-structured network of records [35, 43].	Standard two-dimensional tables (relations) [34, 43].
Relationships	Strictly 1 : N relationships; each child has one parent [36, 43].	Supports complex $M : N$ relationships; records have multiple owners [35, 43].	Highly flexible; handles 1 : 1, 1 : N, and $M : N$ via keys [34, 43].
Data Navigation	Strictly follows hard-coded physical path pointers [43].	Uses complex pointers; require procedural traversal [35, 43].	Logical; based on value matching across columns [43].
Data Independence	Poor; physical structure is coupled with application queries [43].	Poor; pointer structures require rigid schemas [43].	Excellent; supports distinct logical and physical layers [43].

2.2 The Entity-Relationship (ER) Model

Proposed by Peter Chen, the ER model is a high-level conceptual data model used to design database systems prior to implementation [36, 42].

2.2.1 Entity and Entity Sets

An **Entity** is a real-world object (physical or conceptual) that is uniquely distinguishable from other objects [36]. An **Entity Set** is a collection of entities of the same type [36, 37].

2.2.2 Attributes and Classifications

Attributes are properties that describe characteristics of an entity [36].

- ◁ **Simple vs. Composite:** Simple attributes are indivisible (e.g., Age) [23]. Composite attributes can be subdivided into smaller, logical sub-components (e.g., Name splits into FirstName, LastName) [23].

- ◁ **Single-valued vs. Multi-valued:** Single-valued attributes hold at most one value for an entity (e.g., RollNo) [24]. Multi-valued attributes hold a set of values (e.g., Phone Numbers) [24].
- ◁ **Stored vs. Derived:** Stored attributes are physically stored in the database. Derived attributes are computed dynamically from stored attributes (e.g., Age derived from DateOfBirth) [23, 37].

2.2.3 Relationships and Relationship Sets

A **Relationship** is an association among several entities [36]. A **Relationship Set** is a collection of relationships of the same type [38, 39].

2.3 Mapping Constraints

Mapping constraints define the structural rules that govern relationships.

2.3.1 Cardinality Ratios

Cardinality specifies the maximum number of entities that can be associated with an entity via a relationship set [39]:

- ◁ **One-to-One (1:1):** Each entity in *A* is related to at most one entity in *B*, and vice versa [40]. (e.g., Person – Passport) [40].
- ◁ **One-to-Many (1:N):** One entity in *A* can be related to many entities in *B*, but each in *B* is related to at most one in *A* [40]. (e.g., Department – Employee) [40].
- ◁ **Many-to-Many (M:N):** Entities in *A* can relate to many in *B*, and vice-versa [40]. (e.g., Student – Course) [40].

2.3.2 Participation Constraints

Participation constraints define whether the existence of an entity depends on its association with another entity via a relationship set [40]:

- ◁ **Total Participation:** Every entity in the set must participate in at least one relationship instance [41]. Represented by a **double line** [41].
- ◁ **Partial Participation:** Some entities can exist without participating in the relationship [41]. Represented by a **single line** [41].

2.4 Weak Entity Sets

A **Weak Entity Set** is an entity set that does not contain sufficient attributes to construct a primary key on its own [39].

- ◁ **Existential Dependency:** It is existentially dependent on a parent **Identifying (Owner) Entity Set** [39].
- ◁ **Discriminator (Partial Key):** It uses its own partial key (or discriminator) combined with the primary key of the owner entity set to form its primary key [39].
- ◁ **ER Representation:** Drawn using a **double rectangle** with its identifying relationship drawn using a **double diamond** [42]. Its partial key is underlined with a **dashed line** in the diagram.

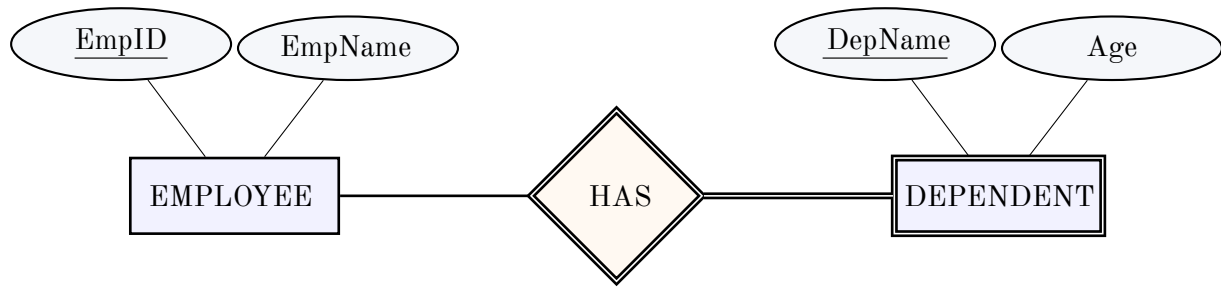


Figure 2.1: Weak Entity Set Represented in an ER Schema Diagram [39, 42]

2.5 ER Diagram Basic Symbols Reference

Table 2.2: Standard ER Notation Reference [41, 42]

Symbol	Represents	Exam Drawing Instructions
Rectangle	Strong Entity Set [41]	Draw a single solid rectangle with the entity set name
Double Rectangle	Weak Entity Set [42]	Draw two concentric rectangles [42].
Diamond	Relationship Set [41]	Draw a single solid diamond with the relationship name
Double Diamond	Identifying Relationship [42]	Draw two concentric diamonds [42]. Used for weak en
Ellipse	Basic Attribute [41]	Draw a single solid ellipse with the attribute name insi
Underlined Ellipse	Primary Key Attribute [42]	Draw a solid ellipse with the attribute name underlined
Dashed Underline	Partial Key (Discriminator)	Underline the attribute name inside the ellipse with a
Double Ellipse	Multi-valued Attribute [42]	Draw two concentric ellipses [42].
Dashed Ellipse	Derived Attribute	Draw a dashed line ellipse [37].
Double Line	Total Participation [41]	Draw parallel double lines connecting entity to relation

2.6 ER Model Design Issues

When modeling databases, designers must avoid common conceptual errors:

- ◁ **Entity Set vs. Attribute:** Decide whether to model an object as an independent entity set or a descriptive attribute of an existing entity [37]. If the object possesses descriptive characteristics of its own or participates in distinct relationships, it must be modeled as an independent entity set [37].
- ◁ **Binary vs. N-ary Relationships:** Binary relationships link exactly two entity sets [38]. N-ary relationships link more than two entity sets [38]. While binary relationships are simpler and more common [38], some scenarios require N-ary relationships to model complex associations without distorting the semantic meaning.

Chapter 3

Relational Algebra and Relational Calculus

3.1 Relational Algebra: Syntax, Semantics, and Basic Operators

Relational Algebra is a **procedural query language** used to retrieve data from relations [44]. It defines a set of operators that take one or more relations as input and produce a new relation as output [44]. It specifies both **what** data to retrieve and **how** to construct the result step-by-step [45].

3.1.1 Unary Operators

Selection (σ)

Filters rows from a relation that satisfy a given selection condition (predicate) [46].

- ◁ **Syntax:** $\sigma_C(R)$ where C is the selection condition and R is the relation [45, 46].
- ◁ **Semantics:** Retrieves all tuples $t \in R$ for which $C(t)$ evaluates to **true** [45].
- ◁ **Example:** Select all student records belonging to the 'CSE' department:

$\sigma_{\text{Department}='CSE'}(\text{STUDENT})$

Projection (π)

Extracts specified attributes (columns) from a relation and automatically eliminates duplicate tuples [46].

- ◁ **Syntax:** $\pi_{A_1, A_2, \dots, A_n}(R)$ where A_i are attribute names [46].
- ◁ **Semantics:** Returns a new relation containing only the projected columns [46].
- ◁ **Example:** Extract only the names and departments of students:

$\pi_{\text{Name, Department}}(\text{STUDENT})$

Rename (ρ)

Renames a relation or its attributes [47].

- ◁ **Syntax:** $\rho_S(R)$ renames relation R to S [47]. $\rho_{S(B_1, \dots, B_n)}(R)$ renames relation R to S and its attributes to $B_1, \dots, B_n$ [47].

3.1.2 Binary Set Operators

For set operations (Union, Set Difference, Intersection), the relations must be **union-compatible**:

1. They must have the same number of attributes (same degree) [46].
2. The domains of the corresponding attributes must be identical (same data types in order) [46].

Union ($\cup$)

Returns a relation containing all tuples from relation R , S , or both, with duplicates removed [46].

◁ **Syntax:** $R \cup S$ [46]

Set Difference ($-$)

Returns tuples present in relation R but not in relation S [47].

◁ **Syntax:** $R - S$ [47]

Cartesian Product ($\times$)

Combines each tuple of relation R with every tuple of relation S [47].

◁ **Syntax:** $R \times S$ [47]

◁ **Degree/Cardinality:** If R has degree D_1 and cardinality C_1 , and S has degree D_2 and cardinality C_2 , then $R \times S$ has degree $(D_1 + D_2)$ and cardinality $(C_1 \times C_2)$.

3.2 Additional Relational Algebra Operators

3.2.1 Join Operations

Theta Join ($\bowtie_C$)

Combines related tuples from two relations based on a logical join condition C [47].

◁ **Syntax:** $R \bowtie_C S = \sigma_C(R \times S)$ [47]

Equi-Join

A special case of the Theta Join where the condition C consists solely of equality comparisons [47].

Natural Join ($\bowtie$)

Joins two relations on all shared attributes, automatically matching equal values and removing duplicate columns [48].

◁ **Syntax:** $R \bowtie S$ [48]

3.2.2 Division Operator ($\div$)

The division operator is used for queries that involve the logical phrase “for all” or “every” [48].

- ◁ **Syntax:** $R \div S$ [48]
- ◁ **Semantics:** Returns a relation over attributes $(A - B)$ where A is the schema of R and B is the schema of S . A tuple t is in $R \div S$ if, for every tuple $s \in S$, the concatenated tuple $t \cdot s$ is present in R .

3.2.3 Grouping and Aggregation Operator (γ)

Used to group tuples by specific attributes and apply aggregate functions [48, 49].

- ◁ **Syntax:** $G_1, G_2, \dots, G_k \gamma F_1(A_1), F_2(A_2), \dots, F_m(A_m)(R)$ [49]
- ◁ **Where:**
 - G_i are the grouping attributes (placed as a left subscript prefix) [49].
 - $F_j(A_j)$ are the aggregate functions (such as COUNT, SUM, AVG, MIN, MAX) applied to attributes A_j (placed as a right subscript suffix) [49].
- ◁ **Example:** Find the average CGPA for each department:

Department γ AVG(CGPA)(STUDENT)

3.3 Introduction to Relational Calculus

Relational Calculus is a **non-procedural (declarative)** query language based on predicate logic [50, 52]. Unlike relational algebra, it describes **what data is required** without specifying the operational steps needed to retrieve it [50, 52].

3.3.1 Logic Operators and Quantifiers Key

To formulate calculus expressions correctly, students must understand the standard mathematical operators:

- ◁ $\exists$ (Existential Quantifier): “There exists” (true if at least one value satisfies the condition) [55].
- ◁ $\forall$ (Universal Quantifier): “For all” (true if all values satisfy the condition) [48].
- ◁ $\wedge$ (Conjunction): “Logical AND” (true if both conditions are true) [51].
- ◁ $\vee$ (Disjunction): “Logical OR” (true if at least one condition is true).
- ◁ $\neg$ (Negation): “Logical NOT” (true if the condition is false).

3.3.2 Tuple Relational Calculus (TRC)

TRC evaluates queries using variables that range over entire database tuples [53].

- ◁ **General Form:** $\{t \mid P(t)\}$ where t is a tuple variable and $P(t)$ is a logical formula [53].
- ◁ **Technical Example:** Retrieve the names of students with a CGPA greater than 8:
 $\{t.\text{Name} \mid t \in \text{STUDENT} \wedge t.\text{CGPA} > 8.0\}$

- ◁ **Explanation:** The query returns a relation containing attribute Name from tuple variable t , where t belongs to relation STUDENT and its CGPA attribute is strictly greater than 8.0 [54].

3.3.3 Domain Relational Calculus (DRC)

DRC evaluates queries using variables that range over individual attribute domains (columns) rather than entire tuples [54].

- ◁ **General Form:** $\{\langle x_1, x_2, \dots, x_n \rangle \mid P(x_1, x_2, \dots, x_n)\}$ where x_i represent domain variables [54].
- ◁ **Technical Example:** Retrieve the RollNo and Name of students in the 'CSE' department:
 $\{\langle r, n \rangle \mid \exists d, c (\langle r, n, d, c \rangle \in \text{STUDENT} \wedge d = \text{'CSE'})\}$
- ◁ **Explanation:** The query selects domain variables r (RollNo) and n (Name) such that there exist domain values d (Department) and c (CGPA) forming a valid tuple in relation STUDENT, with d strictly equal to 'CSE' [55].

3.3.4 Safety of Expressions

A relational calculus expression is considered **safe** if it is guaranteed to yield a finite set of tuples using only values that appear in the database domains [51, 55]. Unsafe expressions can generate infinite results and are prohibited by relational compilers [55]. For example, the TRC expression $\{t \mid \neg(t \in \text{STUDENT})\}$ is unsafe because it describes an infinite set of tuples not belonging to the table [55].

3.4 Computational Capabilities and Codd's Theorem

E. Codd formulated a crucial theorem establishing the expressive equivalence of relational database languages [56, 57].

EXAM FOCUS

Codd's Theorem Statement:

Codd's Theorem states that **Relational Algebra and safe Relational Calculus are computationally equivalent in expressive power** [57]. Any database query that can be formulated procedurally using Relational Algebra can also be written declaratively using safe Tuple or Domain Relational Calculus, and vice-versa [57, 58].

Chapter 4

Master Formula & Rule Sheet

4.1 Core Structural Properties

- ◁ Mathematical Definition of a Relation: $R \subseteq D_1 \times D_2 \times \dots \times D_n$ [21].
- ◁ Degree (D): $D = |\text{Attribute Set}|$ (fixed column count) [22]. 20
- ◁ Cardinality (C): $C = |\text{Tuple Set}|$ (dynamic row count) [22].

4.2 Relational Algebra Operators Cheat Sheet

Operator	Symbol	Mathematical Representation / Definition
Selection	σ	$\sigma_C(R) = \{t \in R \mid C(t)\}$ [46].
Projection	π	$\pi_{A_1, \dots, A_k}(R)$ extracts unique column subsets [46].
Union	$\cup$	$R \cup S = \{t \mid t \in R \vee t \in S\}$ (must be union-compatible) [46].
Set Difference	$-$	$R - S = \{t \mid t \in R \wedge t \notin S\}$ (must be union-compatible) [47].
Cartesian Product	$\times$	$R \times S = \{r \cdot s \mid r \in R \wedge s \in S\}$ [47].
Rename	ρ	$\rho_S(R)$ or $\rho_{S(B_1, \dots, B_n)}(R)$ [47].
Theta Join	$\bowtie_C$	$R \bowtie_C S = \sigma_C(R \times S)$ [47].
Natural Join	$\bowtie$	Joins on common attributes; removes duplicate columns [48].
Division	$\div$	$R \div S = \{t \in \pi_{A-B}(R) \mid \forall s \in S (t \cdot s \in R)\}$ [48].
Grouping	γ	$G_1, \dots, G_k \gamma F_1(A_1), \dots, F_m(A_m)(R)$ [49].

4.3 Database Integrity Constraint Rules

1. Entity Integrity Constraint: $\forall t \in R (t[\text{Primary Key}] \neq \text{NULL})$ [30].
2. Referential Integrity Constraint: If FK is a foreign key in referencing relation R_1 referencing primary key PK of referenced relation R_2 , then:

$$\forall t \in R_1 (t[FK] = \text{NULL} \vee t[FK] \in \pi_{PK}(R_2))$$

Chapter 5

Solved Problem Manual

5.1 Problem Category A: Key Identification & Cardinality Analysis

Scenario: Given a relational schema:

ENROLLMENT(StudentID, CourseID, EnrollmentDate, Grade)

Assume there are 100 unique students, 15 unique courses, and 300 total enrollment rows currently present in the database.

Questions:

1. Determine the **Degree** and current **Cardinality** of the ENROLLMENT relation [22].
2. Can the attribute StudentID serve as the primary key of this relation? Justify your answer.

Step-by-Step Solution:

1. **Degree:** Degree represents the count of attributes (columns) in the schema [22]. The columns are StudentID, CourseID, EnrollmentDate, and Grade. Degree = 4

Cardinality: Cardinality represents the count of tuples (rows) stored [22]. Cardinality = 300

2. **Primary Key Analysis:** No, StudentID cannot serve as the primary key on its own. A primary key must be completely unique for every single tuple in the table [28]. Since a single student can enroll in multiple courses, the StudentID attribute will contain duplicate values. To uniquely identify each row, a composite primary key consisting of both {StudentID, CourseID} is required.

5.2 Problem Category B: Relational Algebra Query Construction

Let the university database schema consist of the following relations:

STUDENT(RollNo, Name, DeptID, Age)

DEPARTMENT(DeptID, DeptName)

COURSE(CourseID, CourseName, Credits)

ENROLLMENT(RollNo, CourseID, Grade)

Write relational algebra expressions for the following queries:

5.2.1 Query 1: Retrieve names and departments of all students older than 20.

1. **Step 1 (Filter):** Select student tuples where Age is greater than 20:

$$R_1 = \sigma_{\text{Age} > 20}(\text{STUDENT})$$

2. **Step 2 (Join):** Perform a Natural Join with DEPARTMENT on DeptID: R_2

$$= R_1 \bowtie \text{DEPARTMENT}$$

3. **Step 3 (Project):** Project the required columns:

$$\text{Result} = \pi_{\text{Name}, \text{DeptName}}(R_2)$$

4. **Final Expression:**

$$\pi_{\text{Name}, \text{DeptName}}(\sigma_{\text{Age} > 20}(\text{STUDENT}) \bowtie \text{DEPARTMENT})$$

5.2.2 Query 2 (Nested Queries/Aggregation): Find departments where average student age is greater than 21.

1. **Step 1 (Aggregate):** Group students by DeptID and compute the average age: R_1

$$= \text{DeptID} \gamma \text{AVG(Age)}(\text{STUDENT})$$

2. **Step 2 (Rename for reference):** Rename the aggregate column to AvgAge: R_2

$$= \rho_{Temp}(\text{DeptID}, \text{AvgAge})(R_1)$$

3. **Step 3 (Filter):** Select groups where AvgAge is greater than 21:

$$R_3 = \sigma_{\text{AvgAge} > 21}(R_2)$$

4. **Step 4 (Join & Project):** Join with DEPARTMENT to retrieve the name:

$$\text{Result} = \pi_{\text{DeptName}}(R_3 \bowtie \text{DEPARTMENT})$$

5. **Final Expression:**

$$\pi_{\text{DeptName}}(\sigma_{\text{AvgAge} > 21}(\rho_{Temp}(\text{DeptID}, \text{AvgAge})(\text{DeptID} \gamma \text{AVG(Age)}(\text{STUDENT}))) \bowtie \text{DEPARTMENT})$$

5.2.3 Query 3 (Division Operator): Find the RollNo of students who have enrolled in EVERY course offered.

The phrase “every course” signals the need for the Division operator ($\div$) [48].

1. **Step 1 (Isolate student enrollment):** Project student enrollments onto RollNo and CourseID: $R_1 = \pi_{\text{RollNo}, \text{CourseID}}(\text{ENROLLMENT})$

2. **Step 2 (Isolate all course keys):** Project all available courses onto CourseID: R_2

$$= \pi_{\text{CourseID}}(\text{COURSE})$$

3. **Step 3 (Divide):** Divide the enrollments relation by the course relation to find students matching all courses:

$$\text{Result} = R_1 \div R_2$$

4. **Final Expression:**

$$\pi_{\text{RollNo}, \text{CourseID}}(\text{ENROLLMENT}) \div \pi_{\text{CourseID}}(\text{COURSE})$$

5.3 Problem Category C: Relational Calculus Query Construction

Using the university schema from the previous section, write Tuple Relational Calculus (TRC) and Domain Relational Calculus (DRC) expressions for the following query:

Query: Retrieve the names of all students who belong to the 'CSE' department.

5.3.1 Tuple Relational Calculus (TRC) Formulation:

- ◁ We seek to return a tuple u containing only the student's name: $u.Name$ [53].
- ◁ There must exist a student tuple $s \in STUDENT$ where the name in s matches our output tuple ($u.Name = s.Name$). 23
- ◁ There must exist a department tuple $d \in DEPARTMENT$ with $DeptName = 'CSE'$ and matching IDs ($s.DeptID = d.DeptID$).
- ◁ **Formula:**

$$\{u \mid \exists s \in STUDENT (u.Name = s.Name \wedge \exists d \in DEPARTMENT (s.DeptID = d.DeptID \wedge d.DeptName =$$

5.3.2 Domain Relational Calculus (DRC) Formulation:

- ◁ Let the domain variables for $STUDENT$ columns be $\langle r, n, di, a \rangle$ corresponding to RollNo, Name, DeptID, and Age.
- ◁ Let the corresponding variables for $DEPARTMENT$ be $\langle d_id, d_name \rangle$.
- ◁ We seek to select the name domain variable $\langle n \rangle$.
- ◁ **Formula:**

$$\{\langle n \rangle \mid \exists r, di, a (\langle r, n, di, a \rangle \in STUDENT \wedge \exists d_id, d_name (\langle d_id, d_name \rangle \in DEPARTMENT \wedge d_name = 'CSE'))\}$$

5.4 Problem Category D: Referential Integrity Cascades

Given two relations with a foreign key constraint:

DEPARTMENT(DeptID, DeptName)

STUDENT(RollNo, Name, DeptID REFERENCES DEPARTMENT)

Let the tables contain the following initial datasets:

DeptID	DeptName	RollNo	Name	DeptID
D1	CSE	101	Rahul	D1
D2	ECE	102	Ananya	D2

Analyze the final database state if you attempt to delete the department 'D1' under:

1. RESTRICT / NO ACTION [32]
2. CASCADE [32]
3. SET NULL [32]

Step-by-Step Solution:

1. **RESTRICT / NO ACTION:** The DBMS checks for referencing rows in the STUDENT table [32]. It finds student 101 (Rahul) referencing department 'D1'. The deletion of department 'D1' is blocked and fails, throwing a referential integrity violation error [32].
2. **CASCADE:** The department row 'D1' is deleted from the DEPARTMENT table [32]. The deletion propagates to all referencing student records [32]. **Final State:**
 - ◁ DEPARTMENT: Contains only (D2, ECE).
 - ◁ STUDENT: Contains only (102, Ananya, D2). (Rahul's record is automatically deleted) [33].
3. **SET NULL:** The department row 'D1' is deleted from the DEPARTMENT table [32]. The foreign key value in student records referencing 'D1' is set to NULL [32]. **Final State:**
 - ◁ DEPARTMENT: Contains only (D2, ECE).
 - ◁ STUDENT: Contains (101, Rahul, NULL) and (102, Ananya, D2) [32].

CU UPDATES

Chapter 6

Expected Exam Question Bank

6.1 Section A: High-Yield 2-Mark Questions

6.1.1 Q1: Define a Database Management System (DBMS). [59]

Answer: A DBMS is a specialized software package that acts as an interface between application programs/end-users and the physical database files, providing systematic operations to define, create, query, maintain, and control access to data [5].

6.1.2 Q2: Differentiate between a Database Schema and a Database Instance. [59]

Answer: A database schema is the structural logical design blueprint of the database which remains stable and changes very rarely [25, 26]. A database instance is the actual dataset populated within that schema at a specific snapshot in time, changing dynamically with daily transactions [25, 26].

6.1.3 Q3: State the minimality property of a Candidate Key. [27]

Answer: The minimality property states that no proper subset of the attributes forming the candidate key can uniquely identify the tuples in the relation [27]. If any attribute is removed from a candidate key, the remaining subset will lose the uniqueness property [27].

6.1.4 Q4: What is the purpose of the Entity Integrity Constraint? [30]

Answer: The Entity Integrity Constraint ensures that no primary key column can contain a NULL value and every primary key value must be unique, guaranteeing that every individual entity remains uniquely identifiable [30].

6.1.5 Q5: Explain why Physical Data Independence is easier to achieve than Logical Data Independence. [11]

Answer: Physical data independence only requires updating the conceptual-to-internal mapping without altering logical table schemas [10, 16]. Logical data independence requires updating the external-to-conceptual mappings after restructuring tables (e.g., table splits during normalization), which is much more complex because it directly affects the exact logical names and columns referenced by application queries [10, 16, 17].

6.1.6 Q6: What does the Division operator ($\div$) in Relational Algebra represent? [48]

Answer: The division operator is used for queries involving the logical phrase “for all” or “every” [48]. It divides a relation R of degree $(m + n)$ by a relation S of degree n , yielding a relation of degree m containing tuples from R that are associated with *every* tuple in S [48].

6.1.7 Q7: Define a Weak Entity Set. Give one real-world example. [39]

Answer: A weak entity set is an entity set that does not contain sufficient attributes to construct a primary key on its own [39]. It is existentially dependent on a parent identifying entity set [39]. Example: **DEPENDENT** of an employee, identified by the owner’s primary key **EmpID** combined with its partial key **DepName** [39].

6.1.8 Q8: Under what condition are two relations Union-Compatible? [46]

Answer: Two relations are union-compatible if: 1. They have the same number of attributes (same degree) [46]. 2. The domains of their corresponding attributes are identical in order (matching data types) [46].

6.2 Section B: Synthesis & Design 5-Mark Questions

6.2.1 Q1: Explain the ANSI-SPARC Three-Level Architecture. Discuss how it supports Data Independence. [62]

Answer:

◁ The Three Levels:

1. **External Level:** Tailored user views of specific database subsets [9, 10].
2. **Conceptual Level:** Global logical structure defining all tables, keys, and constraints [10].
3. **Internal Level:** Physical storage layouts, file organizations, and indexes [10].

◁ Data Independence Mechanisms:

- **Logical Data Independence:** Supported by updating the **External-to-Conceptual Mapping** [10, 16]. When the conceptual logical schema is altered (e.g., adding an attribute), only this mapping is adjusted, leaving existing external views and application queries completely unaffected [10, 16, 17].
- **Physical Data Independence:** Supported by updating the **Conceptual-to-Internal Mapping** [10, 16]. When the physical storage layout changes (e.g., adding a performance index), only this mapping is updated, keeping logical conceptual tables completely unchanged [11, 16].

6.2.2 Q2: Describe the Entity-Relationship (ER) Model attribute classifications. Give clear examples of each. [36]

Answer:

1. Simple vs. Composite Attributes:

◁ **Simple:** Atomic and indivisible. Example: Age [23].

- ◁ **Composite:** Subdivided into smaller logical sub-components. Example: Address into Street, City, ZipCode.

2. Single-valued vs. Multi-valued Attributes:

- ◁ **Single-valued:** Holds a single value per entity instance. Example: AadhaarNo [24].
- ◁ **Multi-valued:** Holds a set of values per instance. Example: EmailIDs or Phone -

3. Stored vs. Derived Attributes:

- ◁ **Stored:** Physically saved. Example: DateOfBirth.
- ◁ **Derived:** Computed dynamically. Example: Age derived from current date and DateOfBirth [23, 37]. Drawn as dashed ellipses [37].

6.2.3 Q3: Compare the Relational, Hierarchical, and Network Data Models. [63]

Answer: Refer to Chapter 2, Section 1 for the comprehensive technical comparison matrix. Highlight the relational model's superior data independence, value-based logical links, and robust mathematical foundation over pointer-based network/hierarchical models [43].

6.3 Section C: Long-Answer & Problem-Solving Questions

6.3.1 Q1: Consider the relational schema:

STUDENT(RollNo, Name, DeptID, Age)
 DEPARTMENT(DeptID, DeptName)
 COURSE(CourseID, CourseName, Credits)
 ENROLLMENT(RollNo, CourseID, Grade)

Write Relational Algebra expressions to:

1. Find the names of students enrolled in CourseID 'CS101'.
2. Retrieve Department Names along with the total number of students in each department.
3. Find the Name and Age of students who have NOT enrolled in any course.

Answer:

1. **Query 1:** Join STUDENT and ENROLLMENT, filter on CourseID = 'CS101', and project Name: $\pi_{\text{Name}}(\sigma_{\text{CourseID}='CS101'}(\text{STUDENT} \bowtie \text{ENROLLMENT}))$
2. **Query 2:** First, perform Natural Join to link tables, then apply grouping and count:
 $\text{DeptName} \gamma \text{COUNT(RollNo)}(\text{STUDENT} \bowtie \text{DEPARTMENT})$
3. **Query 3:** Find RollNo of students with no enrollments using Set Difference, then join to get details:

$$\begin{aligned}
 R_{\text{All}} &= \pi_{\text{RollNo}}(\text{STUDENT}) \quad R_{\text{Enrolled}} = \pi_{\text{RollNo}}(\text{ENROLLMENT}) \quad R_{\text{NotEnrolled}} = \\
 &R_{\text{All}} - R_{\text{Enrolled}} \\
 \text{Result} &= \pi_{\text{Name, Age}}(R_{\text{NotEnrolled}} \bowtie \text{STUDENT})
 \end{aligned}$$

Chapter 7

1-Hour MST Revision Sheet

7.1 TOP 10 MUST-PREPARE QUESTIONS

Mark these off as you complete your final-hour review. Focus on these high-yield topics to secure maximum marks:

- [] 1. **File System vs. DBMS:** Be ready to draw the 5-point comparison table [6].
- [] 2. **Data Independence:** Distinguish Logical vs. Physical. Explain which mapping layer supports each, and why physical is easier [10, 11, 16].
- [] 3. **ANSI-SPARC Levels:** Memorize the drawing of the 3 levels and mappings [9, 10].
- [] 4. **Candidate Key vs. Primary Key:** Be clear on the minimality and NULL-value rules [27, 28].
- [] 5. **Entity & Referential Integrity:** Explain primary key uniqueness (no NULLs) and foreign key referential rules [30, 31].
- [] 6. **Weak Entity Set:** Draw the EMPLOYEE-DEPENDENT ER diagram with double outlines [39, 42].
- [] 7. **ER Attribute Map:** Review Simple, Composite, Multi-valued, and Derived attribute shapes [23, 24, 37].
- [] 8. **Relational Algebra Joins:** Be prepared to write equations for Natural Join ($\bowtie$) and Theta Join ($\bowtie_C$) [47, 48].
- [] 9. **Division Operator ($\div$):** Formulate a query using the division operator to solve a "for all/every" query [48].
- [] 10. **Codd's Theorem:** State the computational equivalence of relational algebra and safe relational calculus [57].

7.2 Final Core Concept Checklist

- ◁ **Data vs. Info:** Raw facts [3] vs. processed actionable insight [3].
- ◁ **DBMS Advantage:** Centralized management mitigates data redundancy and data inconsistency [6, 7].
- ◁ **3-Tier Architecture:** Presentation (client) $\leftrightarrow$ Application Server (logic) $\leftrightarrow$ Database Server (DBMS) [9].

- ◁ **Candidate Key:** Unique and minimal [27].
29
- ◁ **Primary Key:** Unique candidate key chosen by designer; cannot be NULL (Entity Integrity) [28, 30].
- ◁ **Foreign Key:** References a candidate key; can contain duplicate and NULL values [29, 31].
- ◁ **Referential Actions:** CASCADE propagates modifications; SET NULL clears foreign keys; RESTRICT blocks deletions [32].
- ◁ **Weak Entity:** Lacks primary key; existentially dependent on owner entity; identified by partial key [39].
- ◁ **Relational Algebra:** Procedural; basic operators ($\sigma, \pi, \cup, -, \times, \rho$) [44, 46].
- ◁ **Relational Calculus:** Non-procedural; logic-based; TRC (tuples) vs. DRC (domains) [50, 53, 54].

CU UPDATES